



南开大学
Nankai University

南开大学
编译系统原理 实验报告

Lab 1: 编译器预处理、LLVM IR 与
RISC-V

成员 A (LLVM IR): 李培涛 / 2411041

成员 B (RISC-V): 由队友提交前填写

专业: 计算机科学与技术

2026 年 9 月 29 日

目 录

总分工说明	2
成员 A 独立报告 (LLVM IR)	3
1 摘要	3
2 引言	3
2.1 实验背景	3
2.2 本文范围与小组分工	3
2.3 报告结构	3
3 第一部分：语言处理系统完整工作过程	3
3.1 研究对象与阶段总览	3
3.2 预处理器做了什么	4
3.3 编译器做了什么	4
3.4 汇编器做了什么	5
3.5 链接器做了什么	5
3.6 端到端过程的验证边界	5
4 第二部分：LLVM IR 与汇编语言实验	5
4.1 实验环境	5
4.2 示例程序逻辑	6
4.3 源程序	6
4.4 预处理与 LLVM IR 生成	7
4.5 LLVM IR 结构分析	9
4.6 对比实验与边界实验	11
5 第三部分：MLIR 中端处理过程（进阶要求）	12
5.1 任务目标与层级组织	12
5.2 VecAdd 的逐层 lowering 观察点	12
5.3 命令模板与结果记录	13
5.4 进阶实验的验证边界	13
6 结果与讨论	14
6.1 已获得的结果	14
6.2 提交前缺口与责任边界	14
6.3 验证边界	14
7 结论	15
成员 A 附录：复现实验命令	16
成员 A 提交前检查	16

总分工说明

本报告由两名成员分别完成两份独立实验报告后拼接而成。成员 A 负责预处理、LLVM IR 生成、LLVM IR 验证和 LLVM IR 结构分析；成员 B 负责 RISC-V 汇编生成、汇编/链接和目标程序执行分析。除各自负责的后端表示外，实验目的、环境、源程序、编译流程、对比实验、结果讨论和结论等公共部分也由两名成员分别撰写。成员 A 信息已填写，成员 B 信息由队友提交前补充。当前文件仅保留成员 A 的报告正文。

成员 A 独立报告（LLVM IR）

1 摘要

本实验以一个覆盖多种 SysY 语言特性的示例程序为对象，重点研究从源程序到 LLVM 中间表示（LLVM IR）的转换过程。实验首先使用 Clang 的预处理选项观察运行库声明如何注入源程序；随后使用 Clang 生成面向目标平台的 LLVM IR，并用 LLVM 验证器检查模块结构是否合法。报告进一步从全局对象、局部变量、数组寻址、循环、条件分支、函数调用和运行库声明等角度，将源代码与 LLVM IR 指令逐项对应。

示例程序先将数组中的每个元素乘以 2，再求正数之和与正数个数，最后根据整数平均值返回分类结果。已有实验记录中，主机参考程序输出为 28 3 1，LLVM IR 验证通过。本文只报告预处理、LLVM IR 生成、LLVM IR 验证和 IR 结构分析；RISC-V 汇编、汇编器和目标程序部分由小组另一位成员负责。

关键词：编译器；预处理器；LLVM IR；SysY；中间表示；控制流；数组寻址

2 引言

2.1 实验背景

编译器将高级语言程序转换为目标机器可以执行的程序。这个过程不是单次文本替换，而是由预处理、词法分析、语法分析、语义分析、中间表示构造、优化、代码生成、汇编和链接等阶段组成。LLVM 将中间表示作为编译器前端和后端之间的接口：前端负责把源程序的结构和语义编码到 LLVM IR，后续优化与代码生成则以 IR 为输入。LLVM IR 的指令、类型、基本块和控制流关系可以直接观察，因此适合用来理解源程序的语义如何被编译器表示。

本实验使用 SysY 子集程序作为输入。SysY 的语法比完整 C 语言更小，但包含常量、变量、数组、表达式、条件、循环、函数和运行库调用等编译器实验需要的核心结构。示例程序不直接包含主机输入输出头文件，而是通过 `sysy_runtime.h` 声明 `putint` 和 `putch`，从而把语言特性和运行库实现分开。

2.2 本文范围与小组分工

本报告由本人负责 LLVM IR 部分，覆盖预处理器观察、Clang 生成 IR、`opt` 验证 IR、IR 指令阅读和对比实验。小组另一位成员负责 RISC-V 汇编、汇编器、链接器和目标程序部分；这些内容不在本文展开，以免把尚未由本人完成的工作写成个人实验结果。实验任务和报告结构依据课程课件确定。

2.3 报告结构

第 1 部分说明预处理器、编译器各阶段、汇编器和链接器的完整工作过程；第 2 部分记录 SysY 示例、LLVM IR 生成与验证、IR 结构分析和对比实验；第 3 部分单独记录 MLIR 中端 `VecAdd` 的逐层 lowering 观察；最后给出结果讨论、结论、复现命令和提交检查。

3 第一部分：语言处理系统完整工作过程

3.1 研究对象与阶段总览

本部分以 Clang/LLVM 工具链为研究对象，说明一个 C/SysY 风格源程序如何经过预处理、编译、汇编和链接，最终形成可执行目标。命令行工具可以把完整过程拆开观察：`clang -E` 停在预处理结果，`clang -S -emit-llvm` 输出 LLVM IR，`clang -S` 输出汇编，汇编器把汇编文本转换为目标文件，链接器再把目标文件和运行库组合为可执行文件。

表 1: 语言处理系统各阶段及其主要产物

阶段	主要功能	典型产物
预处理	展开头文件、宏和条件编译，形成编译器前端的输入	.i
词法分析	将字符流切分为关键字、标识符、常量和运算符等记号	token 流
语法分析	根据文法构造抽象语法树并报告语法错误	AST
语义分析	检查类型、作用域、返回值和控制流约束	带类型的 AST
IR 生成与优化	把 AST 转换为中间表示并进行目标无关优化	LLVM IR
代码生成	结合目标架构选择指令、寄存器和栈布局	汇编文本
汇编	编码指令、建立符号表并生成重定位信息	.o
链接	合并目标文件、解析外部符号并完成重定位	ELF/可执行文件

3.2 预处理器做了什么

预处理器在编译器正式解析前处理文本级指令。头文件包含会把声明和宏定义插入当前翻译单元，宏展开会替换标识符，条件编译则根据宏环境保留或删除代码。预处理阶段通常不建立完整的类型和控制流语义，因此 `clang -E` 的输出仍然是源代码文本，只是已经完成了这些文本变换。

本实验使用 `-include sysy_runtime.h` 强制注入 `putint` 和 `putch` 的声明。预处理结果用于确认运行库接口确实进入了翻译单元；声明是否匹配调用参数，则由后续语义分析和类型检查负责。

Listing 1: 观察预处理阶段

```
1 clang -E -std=c11 -include sysy_runtime.h \
2   SysY_example.c -o build/SysY_example.i
```

3.3 编译器做了什么

3.3.1 词法分析与语法分析

词法分析把字符序列转换为记号，例如 `int`、函数名、整数常量、数组下标和关系运算符。语法分析按照 SysY/C 文法把记号组织成声明、表达式、语句、函数定义和程序整体的抽象语法树。括号不匹配、缺少分号或语句结构不符合文法的错误通常在这两个阶段被报告。

3.3.2 语义分析

语义分析检查标识符是否已声明、变量和函数的类型是否匹配、数组下标是否用于可索引对象、函数返回值是否满足声明，以及整数除法等运算是否满足语言规则。作用域和符号表也在此阶段建立。预处理器不会完成这些检查，因此预处理文件可以生成并不代表源程序已经语义正确。

3.3.3 LLVM IR 生成、验证和优化

Clang 前端把带类型的 AST 转换为 LLVM IR。全局变量、函数参数、基本块、内存访问和控制流都被编码为 IR 对象和指令；`opt -passes=verify` 检查模块结构是否满足 LLVM IR 的约束。优化器可以进行常量传播、死代码删除、控制流简化和内存到寄存器提升，但必须保持程序语义。

3.3.4 目标代码生成

后端根据目标三元组和 ABI 把 LLVM IR 降低为目标架构的指令。这个阶段需要决定寄存器分配、栈帧布局、调用约定、指令选择和分支形式。本文重点分析 LLVM IR；RISC-V 汇编文本和目标程序由小组分工中的另一部分负责，因此不把未实际完成的后端结果写成本报告结论。

3.4 汇编器做了什么

汇编器读取汇编文本，识别指令和伪指令，把指令操作数编码为机器码，同时建立局部和全局符号表。对于尚不能在当前文件内确定的地址，汇编器在目标文件中留下重定位项；外部函数调用因此可以先生成目标文件，待链接器统一解析。典型命令如下，实际架构选项以小组后端实验使用的工具链为准：

Listing 2: 汇编器阶段的命令模板

```
1 riscv64-unknown-elf-as -march=rv64gc -mabi=lp64d \  
2   build/SysY_example.s -o build/SysY_example.o  
3 readelf -h -S -s build/SysY_example.o
```

3.5 链接器做了什么

链接器把一个或多个目标文件、启动代码和运行库合并，按照链接脚本安排代码段、只读数据段、已初始化数据段和未初始化数据段。它解析 `putint`、`putch` 等外部符号，计算跨文件调用地址并应用重定位，最后写出 ELF 或其他目标格式的可执行文件。若符号缺失、ABI 不兼容或链接脚本不匹配，错误会在此阶段出现。

Listing 3: 链接器阶段的命令模板

```
1 riscv64-unknown-elf-gcc -march=rv64gc -mabi=lp64d \  
2   build/SysY_example.o sysy_runtime.c \  
3   -o build/SysY_example.elf  
4 readelf -h -S -s build/SysY_example.elf
```

3.6 端到端过程的验证边界

每个阶段的成功含义不同：预处理成功只表示文本变换完成，IR 验证通过只表示 IR 结构合法，汇编成功只表示当前汇编文本可以编码，链接成功只表示目标文件和外部符号能够组合。只有在目标环境中执行并得到预期输出，才能把整个流水线的行为验证闭环。本文对 LLVM IR 阶段给出完整分析，对 RISC-V 后端只保留流程说明和分工边界。

4 第二部分：LLVM IR 与汇编语言实验

4.1 实验环境

实验在 WSL 环境中完成，仓库已有记录使用 Clang/LLVM 18.1.3。生成 LLVM IR 的源文件和辅助文件位于 `lab1/SysY_example/`：

表 2: 实验文件及作用

文件	作用
SysY_example.c	SysY 子集示例源程序
sysy_runtime.h	putint、putch 的函数声明
sysy_runtime.c	输出函数的最小适配实现
SysY_example.ll	Clang 生成的 LLVM IR 文本模块
SysY_example_exercises.md	IR 阅读和对比实验清单
build_and_verify.sh	自动执行主机编译、IR 生成和 IR 验证的脚本入口

提交报告前, 应将本机实际的 `clang --version`、`opt --version` 和 `llc --version` 输出保存到附录。本文中的版本号来自仓库已有记录, 不替代本次提交的实际环境记录。

4.2 示例程序逻辑

程序定义常量 `scale_factor=2` 和全局变量 `positive_count`。main 创建数组 `[3, -2, 7, 4, -1]`, 然后执行以下三个步骤:

1. `scale_array` 使用 `while` 循环将每个元素乘以 2, 得到 `[6, -4, 14, 8, -2]`。
2. `sum_positive` 遍历数组, 累加正数 $6 + 14 + 8 = 28$, 并将正数个数记录为 3。
3. `classify_average` 计算整数平均值 $28/3 = 9$ 和余数 1。平均值不小于 5 且余数非负, 因此返回分类值 1。

程序最后通过 `putint` 和 `putch` 输出三个整数和换行, 预期输出为:

Listing 4: 示例程序的预期输出

```
1 28 3 1
```

4.3 源程序

下面的源程序覆盖了常量、全局变量、一维数组、数组形参、算术运算、关系运算、逻辑运算、条件分支、循环、`int/void` 函数、函数调用和运行库调用。

Listing 5: SysY 子集示例程序

```

1 const int scale_factor = 2;
2 int positive_count = 0;
3
4 void scale_array(int values[], int length, int factor) {
5     int i = 0;
6     while (i < length) {
7         values[i] = values[i] * factor;
8         i = i + 1;
9     }
10 }
11
12 int sum_positive(int values[], int length) {
13     int sum = 0;
14     int i = 0;
15     positive_count = 0;
16

```

```

17 while (i < length) {
18     if (values[i] > 0) {
19         sum = sum + values[i];
20         positive_count = positive_count + 1;
21     }
22     i = i + 1;
23 }
24
25 return sum;
26 }
27
28 int classify_average(int sum, int count) {
29     if (count == 0) {
30         return 0;
31     }
32
33     int average = sum / count;
34     int remainder = sum % count;
35
36     if (average >= 5 && remainder >= 0) {
37         return 1;
38     }
39     if (average < 0 || !(average >= 5)) {
40         return -1;
41     }
42     return 1;
43 }
44
45 int main() {
46     int values[5] = {3, -2, 7, 4, -1};
47
48     scale_array(values, 5, scale_factor);
49     int sum = sum_positive(values, 5);
50     int category = classify_average(sum, positive_count);
51
52     putint(sum);
53     putch(32);
54     putint(positive_count);
55     putch(32);
56     putint(category);
57     putch(10);
58     return 0;
59 }

```

4.4 预处理与 LLVM IR 生成

4.4.1 预处理器观察

预处理器负责处理头文件包含、宏和条件编译，并输出经过文本预处理的源程序。虽然示例程序没有直接写 `#include <stdio.h>`，编译命令使用 `-include sysy_runtime.h` 强制注入运行库声明：

Listing 6: 生成预处理结果

```

1 cd "/mnt/d/code_warehouse/Curriculum/Compiler Systems/lab1/SysY_example"
2 clang -E -std=c11 -include sysy_runtime.h \

```

```
3 SysY_example.c -o build/SysY_example.i
```

在 build/SysY_example.i 中应能找到以下声明:

Listing 7: 强制包含的运行库声明

```
1 void putint(int value);
2 void putch(int value);
```

这一步只改变编译器前端看到的源文本, 不负责检查程序的控制流和类型语义。函数是否被正确调用, 要到后续的语义分析和 IR 生成阶段才能体现。

4.4.2 生成主机参考结果

先用主机 Clang 编译源程序和最小运行库适配层, 建立可用于对照的输出:

Listing 8: 生成主机参考程序

```
1 clang -std=c11 -Wall -Wextra -Werror \
2   -include sysy_runtime.h SysY_example.c sysy_runtime.c \
3   -o build/SysY_example.host
4 ./build/SysY_example.host
```

仓库已有验证记录中的主机输出为 28 3 1。该结果用于检查源程序的预期语义, 不能替代对 LLVM IR 结构的检查。

4.4.3 生成 LLVM IR

使用 Clang 的 -S -emit-llvm 选项将编译流程停在 LLVM IR 文本阶段:

Listing 9: 生成 LLVM IR

```
1 clang -target riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
2   -std=c11 -O0 -S -emit-llvm -include sysy_runtime.h \
3   SysY_example.c -o SysY_example.ll
```

这里的 -O0 用于保留更接近源程序的局部变量和基本块, 适合初次阅读。生成模块的开头包含目标数据布局和目标三元组; 这些信息说明 IR 将按照目标平台的指针宽度、ABI 和数据布局解释, 但本报告不展开后续汇编生成。

Listing 10: LLVM IR 模块头和全局对象

```
1 ; ModuleID = 'SysY_example.c'
2 source_filename = "SysY_example.c"
3 target triple = "riscv64-unknown-unknown-elf"
4
5 @scale_factor = dso_local constant i32 2, align 4
6 @positive_count = dso_local global i32 0, align 4
7 @__const.main.values = private unnamed_addr constant [5 x i32]
8   [i32 3, i32 -2, i32 7, i32 4, i32 -1], align 4
```

常量 scale_factor 被放在只读全局对象中; positive_count 是可写全局对象; main 中的数组初值被保存为一个私有常量数组, 进入函数后再复制到局部数组。

4.4.4 验证 LLVM IR

生成文本 IR 后, 用 LLVM 验证器检查模块是否满足 IR 的结构约束:

Listing 11: 验证 LLVM IR

```
1 opt -passes=verify SysY_example.ll -disable-output
```

命令没有标准输出且返回码为 0 时, 表示验证器没有发现结构错误。验证器只检查 IR 的合法性, 不证明程序在所有输入上的算法语义正确; 语义检查仍需要结合源程序、边界输入和运行结果。

4.5 LLVM IR 结构分析

4.5.1 函数参数与局部变量

在 -O0 生成的 IR 中, 函数参数和局部变量通常先通过 `alloca` 建立栈槽, 再用 `store` 初始化、用 `load` 读取。例如 `scale_array` 的开头可以概括为:

Listing 12: `scale_array` 的参数与局部变量初始化

```
1 define dso_local void @scale_array(ptr noundef %0,
2   i32 noundef signext %1, i32 noundef signext %2) {
3   %4 = alloca ptr, align 8
4   %5 = alloca i32, align 4
5   %6 = alloca i32, align 4
6   %7 = alloca i32, align 4
7   store ptr %0, ptr %4, align 8
8   store i32 %1, ptr %5, align 4
9   store i32 %2, ptr %6, align 4
10  store i32 0, ptr %7, align 4
11  br label %8
12 }
```

四个栈槽分别保存数组首地址、数组长度、缩放因子和循环变量 `i`。`alloca` 只分配地址空间; 实际的值传递通过 `load` 和 `store` 完成。优化级别提高后, LLVM 可能把部分变量提升到寄存器或合并基本块, 因此 -O0 与 -O2 的 IR 形态会明显不同。

4.5.2 while 循环和条件分支

`scale_array` 的循环条件被表示为比较指令和条件分支:

Listing 13: `scale_array` 的循环条件

```
1 8:
2   %9 = load i32, ptr %7, align 4
3   %10 = load i32, ptr %5, align 4
4   %11 = icmp slt i32 %9, %10
5   br i1 %11, label %12, label %26
```

当 %11 为真时进入循环体 %12, 否则跳到 %26 返回。循环体最后通过无条件分支回到 %8, 构成控制流回边。源程序中的 `while (i < length)` 因此被拆成了“循环头—循环体—结束块”三个基本结构。

`sum_positive` 还包含嵌套的 `if`。循环头 %7 检查 `i < length`, 元素判断块 %11 检查 `values[i] > 0`, 正数路径 %18 更新 `sum` 和 `positive_count`, 汇合块 %28 递增 `i` 并回到循环头。

4.5.3 数组寻址与 getelementptr

数组形参 `int values[]` 在 LLVM IR 中表现为指针参数, 因为函数接收的是数组首元素地址而不是整个数组副本。访问 `values[i]` 时, IR 先把 32 位下标扩展为索引宽度, 再使用 `getelementptr` 计算元素地址:

Listing 14: 数组元素寻址

```
1 %13 = load i32, ptr %6, align 4
2 %14 = sext i32 %13 to i64
3 %15 = getelementptr inbounds i32, ptr %12, i64 %14
4 %16 = load i32, ptr %15, align 4
```

`getelementptr` 不读取内存, 它只根据元素类型和索引计算地址。真正读取元素的是 `load`。写回元素时使用同样的地址计算过程, 再由 `store` 将乘法结果写入该地址。由于 `i32` 占 4 字节, 目标后端会据此计算元素偏移; 该报告只分析其 LLVM IR 来源。

4.5.4 算术、关系和逻辑运算

示例程序中不同运算在 LLVM IR 中有明确对应:

表 3: 源代码运算与 LLVM IR 指令对应

源代码运算	LLVM IR 表达
<code>+</code> 、 <code>-</code> 、 <code>*</code>	<code>add</code> 、 <code>sub</code> 、 <code>mul</code>
<code>/</code> 、 <code>%</code>	<code>sdiv</code> 、 <code>srem</code>
<code><</code> 、 <code>></code> 、 <code>==</code> 、 <code>>=</code>	<code>icmp slt</code> 、 <code>icmp sgt</code> 、 <code>icmp eq</code> 、 <code>icmp sge</code>
<code>&&</code>	前一个条件成立后进入后一个条件的基本块
<code> </code> 、 <code>!</code>	通过多个比较结果和分支块表达短路逻辑

逻辑运算不一定表现为一个名为 `and` 或 `or` 的算术指令。对于含有短路语义的表达式, Clang 会使用多个基本块和 `br` 指令保证求值顺序。`classify_average` 中的 `count == 0` 保护分支也确保除法和取余只在计数非零时执行。

4.5.5 函数调用、返回值和运行库声明

LLVM IR 使用 `define` 定义函数, 使用 `call` 调用函数, 使用 `ret` 返回。`main` 的调用链为:

Listing 15: main 中的函数调用和运行库调用

```
1 call void @scale_array(ptr noundef %5, i32 noundef signext 5,
2   i32 noundef signext 2)
3 %7 = call signext i32 @sum_positive(ptr noundef %6,
4   i32 noundef signext 5)
5 %10 = call signext i32 @classify_average(i32 noundef signext %8,
6   i32 noundef signext %9)
7 call void @putint(i32 noundef signext %11)
8 call void @putch(i32 noundef signext 32)
```

`putint` 和 `putch` 在当前模块中只有声明, 没有函数体:

Listing 16: 运行库外部声明

```
1 declare dso_local void @putint(i32 noundef signext)
2 declare dso_local void @putch(i32 noundef signext)
```

这体现了编译与链接之间的接口边界。LLVM IR 可以引用外部函数，但外部函数的最终实现由运行库在后续目标构建阶段提供。

4.6 对比实验与边界实验

4.6.1 修改平均值阈值

将源程序中的 `average >= 5` 修改为 `average >= 10`，重新生成 IR：

Listing 17: 重新生成修改后的 LLVM IR

```
1 clang -target riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
2   -std=c11 -O0 -S -emit-llvm -include sysy_runtime.h \
3   SysY_example.c -o build/SysY_example.threshold10.ll
```

重点比较 `classify_average` 中 `icmp sge` 的立即数。阈值从 5 改为 10 后，比较指令的常量也应从 5 变为 10；输入数组不变时平均值仍为 9，因此分类路径应发生变化。最终输出应以实际重新编译和运行结果为准。

4.6.2 优化级别对比

分别以 `-O0` 和 `-O2` 生成 IR：

Listing 18: 比较不同优化级别的 LLVM IR

```
1 clang -target riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
2   -O0 -S -emit-llvm -include sysy_runtime.h \
3   SysY_example.c -o build/SysY_example.O0.ll
4
5 clang -target riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
6   -O2 -S -emit-llvm -include sysy_runtime.h \
7   SysY_example.c -o build/SysY_example.O2.ll
8
9 diff -u build/SysY_example.O0.ll build/SysY_example.O2.ll \
10  > build/SysY_example.O0-O2.diff || true
```

建议至少记录三类差异：局部变量的 `alloca/load/store` 是否减少，循环基本块是否被合并或改写，以及函数属性、常量传播和内存访问是否发生变化。两个优化级别的语义输出应保持一致，但 IR 的结构不必相同。

4.6.3 零长度输入与除零保护

将 `main` 中对 `sum_positive` 的调用改为使用长度 0，或者单独构造 `classify_average(sum,0)` 的测试路径。IR 应保留 `count == 0` 的分支，并在该分支直接返回 0，不会执行 `sdiv` 或 `srem`。这个实验验证的是源程序显式写出的保护逻辑，而不是 LLVM 自动修复了除零错误。

4.6.4 实验记录表

本次已将成员 A 的实验产物保存到 `SysY_example/build/report_results/`，提交时应把该目录与报告一并提交。表 4 记录这些文件对应的实际结果。

表 4: LLVM IR 对比实验记录

实验	应观察的 IR 变化	已保存的结果文件
阈值 5 → 10	icmp sge 立即数由 5 变为 10, 分类输出由 28 3 1 变为 28 3 -1; 见 threshold10.host.out、SysY_example.threshold10.ll	
-00 与 -02	alloca: 17 → 0, load: 36 → 3, store: 25 → 4, 基本块: 17 → 10; 见 SysY_example.00-02.diff	
count=0	输出 0 0 0, IR 保留除零保护分支; 见 zero.host.out、SysY_example.zero.ll	
预处理结果	SysY_example.i 第 12-13 行出现 putint、putch 声明	

5 第三部分：MLIR 中端处理过程（进阶要求）

5.1 任务目标与层级组织

LLVM IR 通常作为一个较低层级的统一中间表示，而 MLIR 允许同一程序在多个方言中逐步降低。VecAdd 示例可以先使用较高层的张量或线性代数操作表达向量加法，再依次降低到循环、向量指令、LLVM 方言和目标相关表示。AscendNPU IR 属于 MLIR 生态中的硬件抽象层；本节记录观察方法和结果表，不把没有在本机工具链中实际运行的 lowering 过程写成已完成结果。

5.2 VecAdd 的逐层 lowering 观察点

建议把每次转换后的文件单独保存，并观察操作名称、类型、显式控制流和内存访问如何变化。一个典型的观察顺序如下：

1. 高层方言：以 linalg.generic、tensor 或课程 VecAdd 方言表达逐元素加法；
2. 循环方言：转换为 affine.for 或 scf.for，显式出现迭代变量和边界；
3. 向量方言：使用 vector.transfer_read、vector.add 和 vector.transfer_write 表达批量数据操作；
4. LLVM 方言：将内存、指针、控制流和函数签名降低为 LLVM dialect 操作；
5. 目标相关阶段：由目标后端或 AscendNPU 相关方言继续完成硬件映射，具体结果取决于课程提供的官方工具链。

Listing 19: VecAdd 高层方言的观察示意

```

1 func.func @vec_add(%a: tensor<1024xf32>,
2                   %b: tensor<1024xf32>) -> tensor<1024xf32> {
3   %c = linalg.generic
4     {indexing_maps = [...], iterator_types = ["parallel"]}
5     ins(%a, %b : tensor<1024xf32>, tensor<1024xf32>)
6     outs(%a : tensor<1024xf32>) {
7     ~bb0(%x: f32, %y: f32, %out: f32):

```

```

8      %sum = arith.addf %x, %y : f32
9      linalg.yield %sum : f32
10 } -> tensor<1024xf32>
11 return %c : tensor<1024xf32>
12 }

```

上面代码只用于说明需要关注的操作层级, 提交时应以官方 VecAdd 示例的实际语法和版本为准。重点不是比较文本长短, 而是确认每一步 lowering 是否保持了输入输出形状、元素类型、迭代范围和逐元素加法语义。

5.3 命令模板与结果记录

以下命令展示一种常见的 MLIR lowering 顺序。不同版本的 pass 名称和依赖可能不同, 执行前应先运行 `mlir-opt --help`, 并记录实际版本。

Listing 20: MLIR VecAdd lowering 命令模板

```

1 mlir-opt --version
2 mlir-opt vecadd.mlir \
3   --one-shot-bufferize \
4   --convert-linalg-to-affine-loops \
5   --lower-affine \
6   --convert-scf-to-cf \
7   --convert-vector-to-llvm \
8   --convert-arith-to-llvm \
9   --convert-func-to-llvm \
10  --reconcile-unrealized-casts \
11  -o build/vecadd_llvm.mlir

```

表 5: MLIR lowering 记录表

层级或 pass	应观察的结构变化	实际文件/结果
VecAdd 高层方言	操作、张量形状和元素类型	未执行: 当前环境无 <code>mlir-opt</code>
<code>linalg</code> 到循环	<code>affine.for</code> / <code>scf.for</code> 和边界	未执行: 需 MLIR 工具链
循环到向量	向量读写和批量加法	未执行: 需 MLIR 工具链
向量到 LLVM 方言	指针、加载/存储和函数签名	未执行: 需 MLIR 工具链
AscendNPU 相关 lowering	硬件抽象层操作及目标接口	未执行: 需 AscendNPU 工具链

5.4 进阶实验的验证边界

只有保存每一步的 IR 文本并用对应工具验证, 才能说明 lowering 过程确实执行。当前 WSL 环境未安装 `mlir-opt`, 也没有配置 AscendNPU 方言工具链, 因此本节命令和表格仍是实验模板; 若要提交进阶实验结果, 需要在具备官方 VecAdd/MLIR 工具链的环境中运行并保存每一步 IR 文件。该缺口不依赖成员 B 的 RISC-V 运行结果。

6 结果与讨论

6.1 已获得的结果

本次在 WSL 中实际运行 Clang/LLVM 18.1.3，以下检查已完成。结果文件统一保存在 `SysY_example/build/report_results/`：

- 主机参考程序输出 `28 3 1`；
- `opt -passes=verify SysY_example.ll -disable-output` 通过；
- Clang 成功生成包含四个用户函数和运行库外部声明的 LLVM IR；
- LLVM IR 中能够定位全局对象、局部栈槽、循环条件、数组寻址、算术指令、条件分支和函数调用。

6.2 提交前缺口与责任边界

目前需要补充的内容按“是否需要另一位成员重新运行程序”分为四类。成员 A 的 LLVM IR 结果已经生成，不需要等待成员 B。当前仓库附带的 RISC-V 工具链和脚本也已经在本机运行成功；但按分工，RISC-V 的独立分析仍由成员 B 负责整理。进阶 MLIR 结果则需要额外的 MLIR/AscendNPU 工具链。

表 6: 提交前缺口、负责人和所需产物

内容	当前状态与缺口	提交时需要的文件
LLVM IR (成员 A)	已由本次实验实际运行并验证；包括预处理、主机输出、O0/O2、阈值变体和零长度变体。	<code>report_results/</code> 中的 <code>.i</code> 、 <code>.ll</code> 、 <code>.out</code> 、 <code>.diff</code>
RISC-V 后端 (成员 B)	仓库脚本已使用现有工具链成功完成汇编、链接和模拟器运行，输出为 <code>28 3 1</code> ；成员 B 仍需把这些产物和独立分析写入其负责部分。	<code>.s</code> 、 <code>.elf</code> 、运行日志或截图
MLIR/AscendNPU 进阶部分	当前环境没有 <code>mlir-opt</code> 和 AscendNPU 方言工具链，无法在本机生成逐层 lowering 结果。	在可用环境中保存每一步 <code>.mlir</code> 、命令输出和版本信息
封面信息	成员 A 的姓名、学号和专业已经填写；成员 B 信息仍由队友补充。	补充成员 B 的姓名和学号

主机输出与静态 IR 分析相互补充：前者说明当前输入上的程序计算结果符合预期，后者说明源程序结构确实被编码为相应的 LLVM IR。单个输入的正确输出不能推出所有输入都正确，因此报告保留阈值、优化级别和零长度输入实验作为进一步检查。

6.3 验证边界

本报告不展开 RISC-V 汇编文本、汇编器命令、目标文件链接和模拟器执行分析，这些内容属于小组另一位成员的报告范围。仓库附带的整链脚本已经在当前环境执行成功，但本文对目标三元组的提及只用于解释 LLVM IR 的数据布局和生成环境，不把后端生成结果作为成员 A 的个人实验结论。提交整组报告时，由成员 B 整理 RISC-V 代码、日志和截图，并按封面分工说明合并提交。

本次运行使用 WSL 中的 Clang/LLVM 18.1.3。主机输出、预处理文件、O0/O2 IR、阈值变体、零长度变体、优化差异文件以及本次 RISC-V 整链运行日志均已保存到 `SysY_example/build/report_results/`；提交时将该目录作为实验结果文件一并提交即可。RISC-V 结果已经可以由当前仓库脚本复现，MLIR/AscendNPU 输出仍需要具备相应工具链的环境。

7 结论

本实验通过一个覆盖数组、循环、条件、函数和运行库调用的 SysY 示例，观察了预处理器如何向源程序注入运行库声明，以及 Clang 如何将源代码转换为 LLVM IR。分析表明，`alloca`、`load`、`store` 表示局部变量和内存读写，`getelementptr` 表示数组元素地址计算，`icmp` 与 `br` 表示条件和控制流，`call` 与 `ret` 表示函数接口，外部 `declare` 则保留了运行库链接边界。

通过阈值修改、-O0/-O2 对比和零长度输入实验，可以把源代码改动与 IR 指令、基本块和保护分支的变化联系起来。本报告对应的 LLVM IR 结果文件已经生成；提交时补充成员 B 的姓名和学号，并将成员 B 的 RISC-V 结果文件合并到整组提交目录。

成员 A 附录：复现实验命令

以下命令均在 lab1/SysY_example 目录执行：

Listing 21: LLVM IR 实验的最小复现命令

```
1 clang --version
2 opt --version
3 llc --version
4
5 clang -E -std=c11 -include sysy_runtime.h \
6     SysY_example.c -o build/SysY_example.i
7
8 clang -target riscv64-unknown-elf -march=rv64gc -mabi=lp64d \
9     -std=c11 -O0 -S -emit-llvm -include sysy_runtime.h \
10    SysY_example.c -o SysY_example.ll
11
12 opt -passes=verify SysY_example.ll -disable-output
```

成员 A 提交前检查

- 确认成员 A 的姓名、学号、专业和分工；
- 保存实际工具版本和完整命令输出；
- 确认 SysY_example/build/report_results/ 与报告、源代码一起放在 lab1 提交目录中；
- 检查 LLVM IR 代码、结果文件路径和实验表格；
- 由成员 B 补入 RISC-V 的汇编、链接和运行产物；若提交进阶要求，再补入 MLIR 每一步 lowering 文件；
- 使用 XeLaTeX 编译本文并检查目录、代码行号和表格。